Simón, con Arduino

Aunque supongo que ya es de sobra conocido, Simón, es un juego electrónico, en el que se han de repetir secuencias de colores, que se muestran mediante luces. Para más información, tenemos la siempre socorrida Wikipedia:

https://es.wikipedia.org/wiki/Simon_(juego)

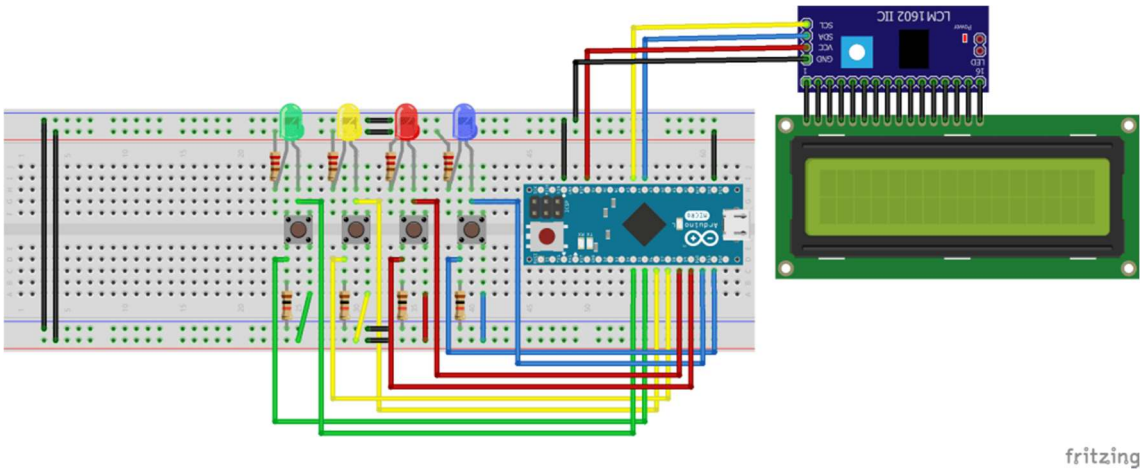
En esta entrada vamos a crear el juego de Simón, controlado con un Arduino Nano.

Se usarán 4 botones y 4 leds como componentes del juego y un display LCD de 16 caracteres y dos líneas para mostrar mensajes sobre el juego.

Componentes:

- Arduino Nano
- 4 leds (1 azul, 1 rojo, 1 amarillo, 1 verde)
- 4 resistencias de 220 ohms
- 4 botones
- 4 resistencias de 10 kohms
- 1 LCD 1602 con módulo LCM 1602 I2C
- Protoboard y cables

Esquema (Fritzing):



Aunque en el esquema se ve el módulo I2C separado del display 1602, se puede comprar todo junto y ya soldado.

Conexiones:

Para los botones se usan resistencias pull down. Una de las patillas se conecta a 3.3v y la otra se conecta a un pin de Arduino, a una resistencia de 10kohms y a masa.

En el esquema, los botones están conectados a los siguientes pines:

Botón	**Pin**
Azul	11
Rojo	9
Amarilo	7
Verde	5

En el esquema, los leds están conectados a los siguiente pines:

Led	**Pin**
Azul	10
Rojo	8
Amarilo	6
Verde	4

El ánodo de los leds se conecta a un pin de Arduino y el cátodo se conecta a una resistencia de 220ohms y a masa.

En el esquema, el módulo I2C se conecta a los siguientes pines:

Módulo I2C	**Pin**
SCL	A5
SDA	A4
VCC	5V
GND	GND

En el tabla se indican los pines para SCL y SDA en Arduino Nano. Se pueden consultar las correspondencias de pines para otros modelos en el siguiente enlace:

https://www.arduino.cc/en/Reference/Wire

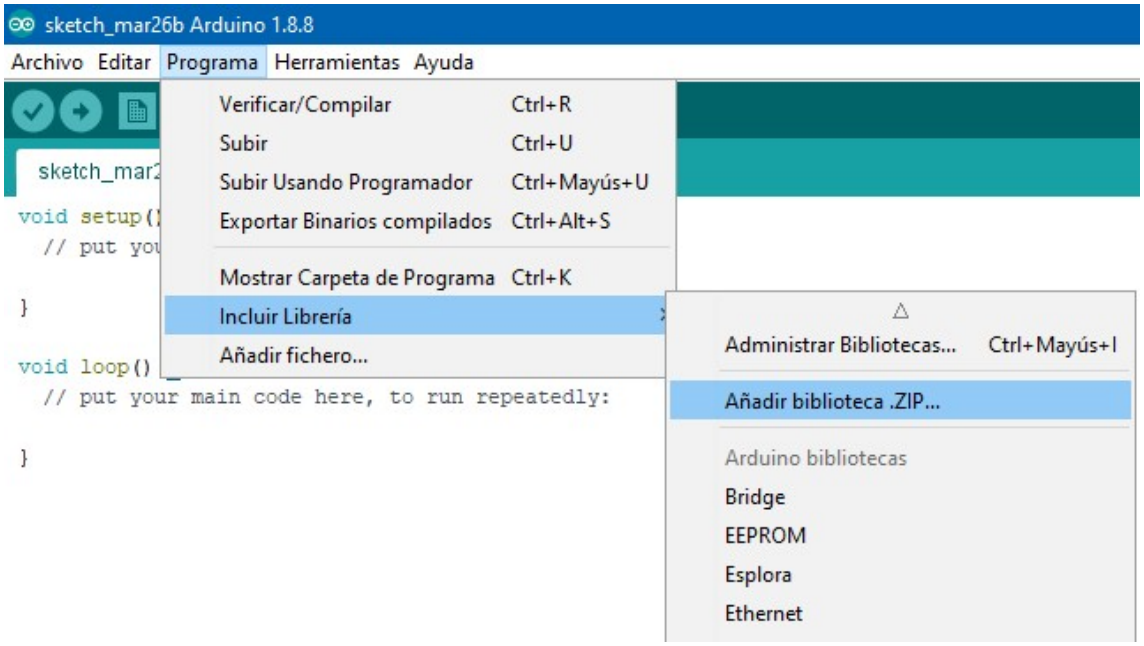
Librería para I2C

Para poder manejar fácilmente el LCD 1602 a través del módulo I2C será necesario importar una librería al sketch.

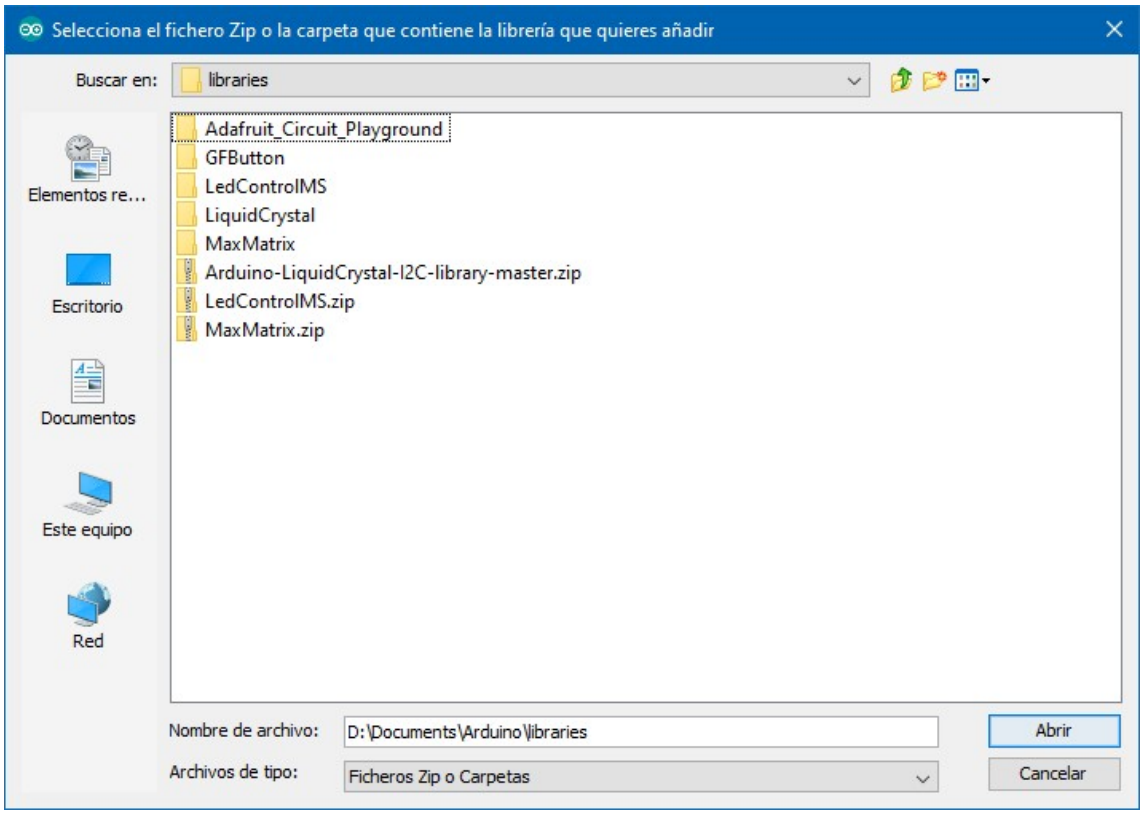
Descargamos la librería desde el siguiente repositorio de Github:

https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

Una vez descargado el fichero zip, se ha de importar la librería a Arduino. Para esto se ha de desplegar el menú **Programa**, submenú **Incluir librería** y, pulsar sobre la opción **Añadir biblioteca .ZIP...**

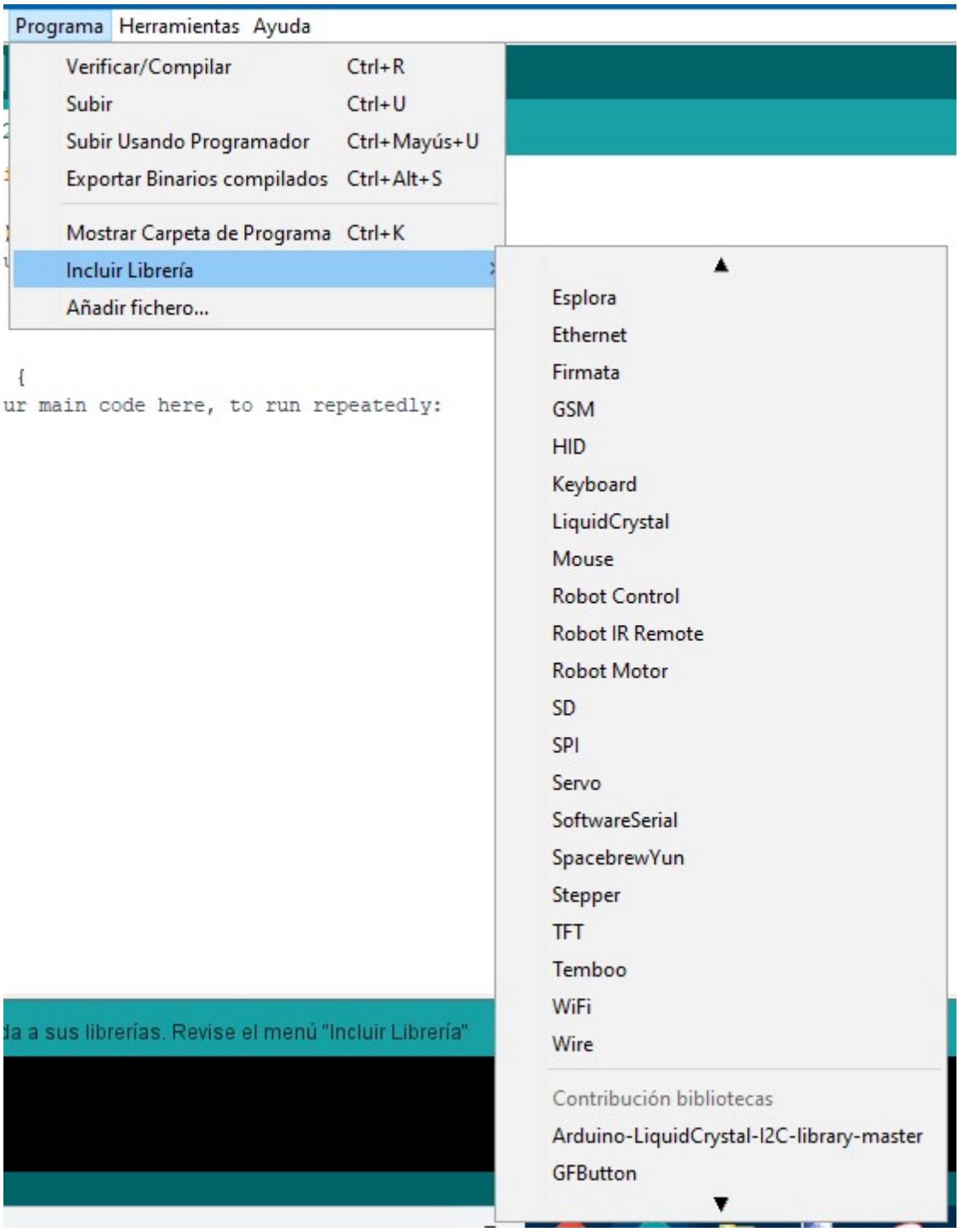


Se abre un cuadro de diálogo para navegar hasta la ubicación del fichero y seleccionarlo.



Al abrir el fichero, se descomprime la librería en el directorio de librerías de Arduino.

Para incluirla en el código se ha de desplegar el menú **Programa**, submenú **Incluir Librería** y, dentro de la sección **Contribución bibliotecas**, pulsar sobre la librería a incluir, en este caso **Arduino-LiquidCrystal-I2C-library-master**.



Al código se añadirá una línea como la siguiente:

```
#include <LiquidCrystal_I2C.h>
```

Seguidamente se ha de instanciar un objeto para poder manejar el LCD usando la librería. Se llama al constructor pasándole los siguientes parámetros:

- dirección de memoria: la dirección de memoria que identifica al dispositivo. Normalmente la 0x27, aunque se puede obtener usando un escáner I2C (https://playground.arduino.cc/Main/I2cScanner/).
- número de caracteres por línea: en este caso 16.
- número de líneas: en este caso 2.

La línea de código quedaría así:

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

Variables globales

Para los leds usaremos cuatro constantes. Para los botones no usaremos constantes, simplemente, sumaremos uno a la constante del led que le corresponda.

```
const byte BLUE = 10;
const byte RED = 8;
const byte YELLOW = 6;
const byte GREEN = 4;
```

Para el control del juego se usarán 3 variables:

- **plays**: Un array para ir almacenando los pasos en la secuencia a repetir. Según el número de elementos que le indiquemos al inicializarlo, se aumentará o disminuirá la dificultad.
- **PlayPosition**: Una variable para guardar la posición en la secuencia en la que se encuentra, para comprobar las jugadas.
- **playLimit**: Una variable para guardar la posición hasta la que debe llegar el jugador para completar la secuencia.

```
bytes plays[25] = {};
byte playPosition = 0;
byte playLimit = 0;
```

Por último, necesitaremos un par de variables boleanas para controlar si el juego ha empezado y para bloquear los botones.

```
boolean bloquedButtons = true;
boolean gameStarted = false;
```

La función setup

La función setup se encargará de iniciar los pins de los leds en modo de salida, y los de los botones en modo de entrada. Inicializará el LCD, mediante una función de la librería I2C y mostrará un mensaje por el display informando de como iniciar el juego.

```
void setup() {
          // LEDS
          pinMode(BLUE, OUTPUT);
          pinMode(RED, OUTPUT);
          pinMode(YELLOW, OUTPUT);
          pinMode(GREEN, OUTPUT);
          // BOTONES
          pinMode(BLUE + 1, INPUT);
          pinMode(RED + 1, INPUT);
          pinMode(YELLOW + 1, INPUT);
          pinMode(GREEN + 1, INPUT);
          // INICIAR EL LCD
          lcd.begin();
          // MOSTRAR MENSAJE DE INICIO
          startMessage();
}
```

La función startMessage se encargará de mostrar un par de líneas en el LCD con el mensaje de inicio.

Para mostrar los mensajes usa la función showMessage, pasándole cuatro parámetros.

- la columna donde ha de colocar el cursor para escribir
- la fila donde ha de colocar el cursor para escribir
- el mensaje a mostrar
- un boleano para indicar si ha de borrar o no el LCD antes de escribir

```
void startMessage() {
          showMessage(0, 0, "Simon The Game", true);
          showMessage(0, 0, "Blue to start", false);
}
void showMessage(byte col, byte row, String message, boolean clearScreen) {
          if(clearScreen) {
                    lcd.clear();
          }
          lcd.setCursor(col, row);
          lcd.print(message);
}
```

La función loop

La función loop se encargará de comprobar si el juego no se ha iniciado, para así comprobar si se pulsa el botón para iniciar nuevo, y, una vez iniciado un juego, comprobará los botones pulsados por el jugador y si lo ha hecho correctamente.

Comprobar si se ha iniciado un juego:

```
if(!gameStarted) {
          checkStartGame();
}

void checkStartGame() {
          if(digitalRead(BLUE + 1) == HIGH) {
                    delay(100);
                    if(digitalRead(BLUE + 1) == LOW) {
                              gameStarted = true;
                              showMessage(0, 0, "Ready?", true);
                              delay(2000);
                              showMessage(0, 0, "Game started!", true);
                              delay(1000);
                              playPosition = 0;
                              playLimit = 0;
                              showSequence();
                    }
          }
}
```

La función checkStartGame comprueba si el botón azul está en HIGH, es decir, pulsado, y espera 100 milisegundos (esto se hace para eliminar rebotes, para más información consultar:

https://www.geekfactory.mx/tutoriales/tutoriales-arduino/boton-o-pulsador-con-arduino/). Seguidamente, comprueba si el botón está ya en LOW (el jugador ha dejado de pulsarlo), para iniciar el juego.

Muestra un par de mensajes para informar al jugador que empieza el juego, reinicia las variables de control del juego y llama a la función showSequence() para mostrar la primera secuencia de luces.

```
void showSequence() {
          randomSeed(millis() / (playLimit + 1));
          byte id = random(4);
          switch(id){
                    case 0:
                              plays[playLimit] = BLUE;
                              break;
                    case 1:
                              plays[playLimit] = RED;
                              break;
                    case 2:
                              plays[playLimit] = YELLOW;
                              break;
                    case 3:
                              plays[playLimit] = GREEN;
                                        break;
          }
          playLimit++;
          if(playLimit > sizeof(plays)/sizeof(plays[0])) {
                    gameStarted = false;
                    showMessage(0, 0, "Congratulations!", true);
                    showMessage(0, 1, "You have win!", false);
                    delay(3000);
                    startMessage();
          } else {
                    showMessage(0, 0, "Play #" + (String)playLimit, true);
                    for(byte i = 0; i < playLimit; i++) {
                              digitalWrite(plays[i], HIGH);
                              delay(1000);
                              digitalWrite(plays[i], LOW);
                              delay(500);
                    }
                    bloquedButtons = false;
          }
}
```

Como primer paso, la función showSequence(), crea una semilla para generar número aleatorios y elige uno del 0 al 3, para almacenarlo en la tabla de pasos de secuencia.

Incrementa el límite de la secuencia y, si este supera al número máximo de pasos, se informa al jugador que ha ganado el juego, si no, se muestra la secuencia de luces que debe repetir el jugador y se liberan los botones.

En caso de ganar el juego, se indica que el juego ha finalizado, mediante la variable gameStarted y se muestra el mensaje de felicitación. Posteriormente, se hace una pausa de 3 segundos, para, seguidamente, mostrar el mensaje de bienvenida mediante startMessage().

Si todavía no se ha finalizado el juego, se muestra un mensaje con el número de paso en la secuencia que se está jugando. Se muestra la secuencia a repetir,

empezando desde el índice 0 de la secuencia de pasos hasta el que indique la variable playLimit. Entre el encendido y el apagado del led en un paso se establece una pausa de un segundo y, entre el apagado y el encendido una de medio segundo.

Al desbloquear los botones, mediante la variable bloqueButtons, ya se pueden comprobar las pulsaciones del jugador, en la función loop().

```
if(!bloquedButtons) {
          byte buttons[4] = { BLUE + 1, RED + 1, YELLOW + 1, GREEN + 1 };
          for(byte i = 0; i < sizeof(buttons) / sizeof(buttons[0]); i++) {
                    if(digitalRead(buttons[i]) == HIGH) {
                              delay(100);
                              digitalWrite(buttons[i] - 1, HIGH);
                              delay(500);
                              digitalWrite(buttons[i] - 1, LOW);
                              delay(500);
                              if(plays[playPosition] == buttons[i] - 1) {
                                        showMessage(0, 1, "Correct!", false);
                                        playPosition++;
                                        if(playPosition == playLimit) {
                                                  playPosition = 0;
                                                  bloquedButtons = true;
                                                  showSequence();
                                        }
                              } else {
                                        showMessage(0, 0, "Error!", true);
                                        showMessage(0, 1, "You have lost!", false);
                                        bloquedButtons = true;
                                        gameStarted = false;
                                        delay(3000);
                                        startMessage();
                              }
                    }
          }
}
```

Se crea un array con los pines de los botones, teniendo en cuenta que la posición de cada botón debe corresponder con el valor que se le ha asignado a su color al generar la secuencia. Es decir, si, cuando al comprobar el número aleatorio para la secuencia, si este era 0 y se ha guardado el color azul, la posición 0 del array de botones, deberá corresponder al botón azul.

Mediante un bucle, recorremos este array y se comprueba si alguno de los botones esta en HIGH. En caso de ser así, se enciende el led del color correspondiente y, pasado medio segundo, se apaga.

Se comprueba si el número del led guardado en la posición de la secuencia que se está jugando es igual al del botón pulsado. Si es así, se incrementa y se comprueba si se ha llegado al final de la secuencia que se está jugando. En caso afirmativo, se inicializa la variable playPosition, se bloquean botones y se muestra la nueva secuencia de luces.

Si se produce un error al pulsar los botones, el juego termina. Se muestra el mensaje informativo, se bloquean botones, se indica que el juego ya no está iniciado, y, después de una pausa de 3 segundos, se vuelve a mostrar el mensaje de inicio.